

マッチング候補者選定ロジック

`src/pages/MatchingPage.tsx / supabase/functions/match-batch/ / auto-match/ / match-score/` の実装に基づく。

概要

AI 呼び出しを 1 案件 = 1 コールに圧縮するため、コミット `b35df40` 「マッチング AI 使用量削減（案A+B）」で **ルールベース事前フィルタ + バッチプロンプト** 方式へ全面再設計された。

```
全候補者（数百人）
├─ RPC fetch_candidates_for_matching（直近30日+経験年数順 800件）
│   └─ duplicate_flag=true と merged_into!=null を除外（クライアント側）
│       └─ match-batch Edge Function
│           ├── calcRuleScore で全員をスコアリング（スキル40 + 経験15 +
│           │   単価15 + 勤務地20 + リモート10 = 100pt）
│           ├── 上位 topN（既定 10 名）だけ AI に再採点を依頼（1 コール）
│           │   ├── Cerebras llama3.1-8b（1段目・20s）
│           │   ├── Groq llama-3.3-70b-versatile（2段目・25s）
│           │   └── Gemini gemini-2.5-flash（最終フォールバック・30s）
│           │       └─ 全段失敗時はルールスコアで全代替
│           └─ (usedModel='rule')
│               └─ 残り（ruleOnly）はルールスコアのみで返す
```

`skill_master` の正規化テーブルでスキルの表記ゆれ（React/React.js/ReactJS など）は内部で同名扱いされる。`auto-match`（毎朝 JST 9:00 cron）は `match-batch` を内部呼び出しするため、同じフォールバック順を継承する（**Gemini 単発はもう使われない**）。

候補者取得 RPC

`fetch_candidates_for_matching(p_data_env, p_limit DEFAULT 800)`

コミット `51f966d` で追加。MatchingPage が「人材→案件」方向で全候補者を取りに行くときに使う。

- `merged_into IS NULL` でフィルタ
- `ORDER BY created_at DESC, COALESCE(experience_years, 0) DESC`
- limit 800（高速モード以外でも上限）
- 「直近 30 日バケツ方式は件数が上限を超えるため不採用」とコメントに明記

`fetch_candidates_for_project(p_data_env, p_skills text[], p_limit DEFAULT 500)`

コミット `a2c0e96` で追加。案件→人材方向の SQL 絞り込み用。

- `candidates.skills` は jsonb 配列のため、PostgreSQL の `&&` 演算子が使えない

- `jsonb_array_elements_text` で展開し、小文字化して `ANY(unnest(p_skills))` を小文字化) で照合
- 結果は `created_at DESC, experience_years DESC` 順、limit 500

ルールベーススコア配点 (`match-batch` の `calcRuleScore`)

観点	配点	ロジック
スキル一致	最大 40pt	<code>required_skills.length > 0</code> のとき：完全一致 1pt / includes 部分一致 0.5pt → $(hits/required.length) * 40$ 。required 空のときは固定 +20pt
経験年数	最大 15pt	10年=15 / 7年=12 / 5年=8 / 3年=4 / 1年=2 / それ以下=0
単価	最大 15pt	<code>budgetMax==null</code> なら +15 固定、範囲内 +15、上限+10% +8、上限+20% +3
勤務地	最大 20pt	<code>prefCore = candPref.replace(/[都道府県]\$/, '')</code> してプロジェクトの <code>workLocation</code> に includes 一致で +20。フルリモート (<code>/フルリモート 完全リモート 100[%%]リモート/</code>) は無条件 +20。居住地不明は +5
リモート	最大 10pt	<code>!isFullRemote && remoteAvailable && /リモート remote 在宅/i.test(remotePolicy)</code> で +10
合計	100pt	—

AI 再採点プロンプト (150 字以内・5 観点優先順)

各候補者に `ruleScore` を埋め込み、AI には次のとおり指示する:

1. 必須スキルの合致状況 (何が合っていて何が不足か)
2. 経験年数と案件要件の比較
3. 単価の合致・乖離
4. 勤務地・リモート希望の一致
5. 懸念点があれば明記

出力形式: `[{"id":"...", "score":整数, "summary":"150字以内"}, ...]`

topN・除外ルール

- `duplicate_flag=true` の候補者は マッチング対象から完全除外 (コミット `1bf49ff`)
- `merged_into != null` も除外 (重複マージ済み)
- `accepted` 状態の人材は `auto-match` 側で除外 (提案済みのため)
- 既存 `submissions` ペアは `auto-match` 側で除外

高速モード vs 全件モード vs auto-match

観点	高速モード (UI)	全件モード (UI)	auto-match (cron)
案件→人材の対象件数	<code>matching_fast_max_candidates</code> (既定 20)	全候補者	案件 1 件あたり最大 40 名 (<code>BATCH_AI_SIZE=20</code> × 2 リクエスト)
人材→案件の対象件数	<code>matching_fast_max_projects</code> (既定 10)	全案件	未対応
AI 採点される件数	topN (既定 10)	topN (既定 10)	topN (既定 10) × 2 リクエスト
残り (ruleOnly) の扱い	スコア順表示	スコア順表示	submissions に upsert

上限件数は `app_config.matching_fast_max_candidates / matching_fast_max_projects` で変更可能 (設定画面のフォームから保存)。

マッチング詳細パネル

コミット `c6ced01` で `MatchingPage` の右ペインに案件サマリーを表示するようになった。

項目	内容
必須スキル	上位 10 件まで (残りは "+ N 件" 表示)
予算	<code>budget_min</code> ～ <code>budget_max</code>
勤務地	<code>work_location</code>
リモート	<code>remote_policy</code>
開始日	<code>start_date</code>
案件概要	<code>roleSummary</code> / <code>description</code> の先頭 150 字

スコア色分け (`scoreColor`) :

- 80 以上: 緑
- 60 以上: 黄
- それ未満: 灰

bulk マッチングの進捗・キャンセル

`MatchingPage` は bulk 実行中に進捗を表示 (`MatchRunProgress`)。

案件 N/M (タイトル) ・ 候補者 N/M 件目 ・ 全体 done/total

- キャンセル機構: `bulkCancelRequestedRef` を立てるとループが停止
- MatchingPage は常時マウント (`hidden` で切替)。タブ切替で mutation が中断されないように設計

エラー収集

MatchingPage の全 mutation の `onError` で `logError(e, 'MatchingPage', undefined, { dataEnv, nickname })` を呼び、`error_logs` テーブルに保存される (コミット `a2c0e96`)。自動マッチ / バルクマッチの失敗を後追い可能。

現状の制約・既知の課題

課題	具体例
スキル文字列の完全/includes 一致	案件「React」、候補者「react.js」→ ルール照合では includes が効くので 0.5pt 加算される
表記ゆれ吸収は skill_master.aliases 任せ	aliases に登録されていれば吸収可、未登録なら別スキル扱い
駅マップの同名衝突	「町田」「野田」「福島」は後勝ち上書きされる (要改善)
業界・役割の重み	スコア配点に直接は含まれず、AI 再採点プロンプトで観点として渡される

関連設定

設定項目	デフォルト	変更場所
高速モード 案件ごとの候補者上限	20 人	設定画面 > マッチング設定 (<code>matching_fast_max_candidates</code>)
高速モード 人材ごとの案件上限	10 件	設定画面 > マッチング設定 (<code>matching_fast_max_projects</code>)
自動マッチング有効	<code>true</code>	設定画面 > 自動マッチング ON/OFF (<code>auto_match_enabled</code>)
即時マッチング (inbound-email 経由)	<code>false</code>	Supabase Secrets <code>AUTO_MATCH_ENABLED</code>

最終更新: 2026-05-23